SP-108 Green ASM

SRS

CS 4850 - Section 01 – Spring 2026

2/8/2026 Professor Perry

Alex Tidwell Team Leader Documentation + Developer (as needed)	Katherine Smith Documentation + Developer	Sumaya Tamenne Documentation	Phillip Eiser Developer	Brianna Lewis Developer

Table of Contents

1.0 Introduction 4

1.1 Overview 4

1.2 Project Scope 4

1.3 Definitions and Acronyms 5

1.4 Assumptions 5

1.5 References 6

2.0 Constraints 6

2.1 Environment 6

2.2 System 7

3.0 Functional Requirements 7

3.1 Asset Discovery 7

3.2 Active Probing 7

3.3 Vulnerability Detection 7

3.3.1 The system shall detect configuration drift and other new vulnerabilities in the target e nvironment. 7

3.3.2 The system shall compare found vulnerabilities with CVE identifiers when available. 7

3.3.3 The system shall store CVE comparisons and vulnerability detections for historical comparison. 7

3.4 Reporting & Notifications 7

3.4.1 The system shall generate attack surface reports and drift reports highlighting any changes sinc e the previous scans. 7

3.4.2 The system shall store all reports in a CSV format file of all drift reports and known CVEs. 7

3.5 Dashboard & GUI 7

4.0 Non-Functional Requirements 8

4.2 Security 8

5.0 Target Environment Requirements 8

5.1 Target Infrastructure 8

5.2 Network Configuration 8

5.3 Vulnerability Injection 8

6.0 External Interface Requirements (use if applicable) 8

6.1 User Interface Requirements 8

6.2 Hardware Interface Requirements 8
6.3 Software Interface Requirements 9
6.4 Communication Interface Requirements 9
7.0 Analysis 9
7.1 Use Cases 9
7.2 Data Flow 10

1.0 Introduction

1.1 Overview

A top priority for all organizations is the security of their assets against cyberattacks. Organizations are changing constantly, whether they are using new apps, cloud services, or contractors. With constant change comes the risk of being exposed to severe attacks. Even with precautions, assets can be accidentally exposed, and a single overlooked entry point can cause consequential damage. For these reasons, Attack Surface Management (ASM) systems provide an additional layer of protection for an organization's assets. An attack surface comprises all exposed digital assets, such as IP addresses and servers, domains and subdomains, APIs, cloud resources, and open ports. This project will implement an ASM system designed to continuously identify, understand, and monitor everything an attacker could see or exploit in an organization's digital environment.

1.2 Project Scope

The primary goal of this project is to design and implement an automated ASM system developed within a controlled environment. The scope is limited to discovering, monitoring, and reporting vulnerabilities and exposed assets, and to detecting changes in the system configuration. This system is a functional prototype and not a fully developed ASM. The following describes the scope of the project:

Within Scope:

- **External Asset Discovery:** This project will identify externally reachable assets, detect new or changed subdomains, and maintain an updated asset inventory with timestamps. Additionally, the system will locate hosts, open ports, services, and subdomains within the controlled environment.
- **Asset Mapping and Inventory Management:** Assets discovered will be correlated with the services and ports they expose and stored in a database with timestamps and information on their status.
- **Continuous Probing and Response Validation:** The system will send probes according to specific protocols to validate service behavior and identify abnormal behavior.
- **CVA Data Ingestion:** The system will ingest data from the NIST National Vulnerability Database (NVD) and store data locally for further analysis.
- **Vulnerability and System Configuration Detection:** The system will detect new vulnerabilities and configuration changes by continuously scanning and comparing results against known CVEs.
- **Threat Monitoring and Red Phase Detection:** The system will identify and record changes made by a simulated attacker (Red Phase).
- **Alerts:** The system will notify users when new vulnerabilities or threats are detected via drift reports. Drift reports will be delivered via supported notification channels, such as Discord.

Out of Scope:

- **Active Exploitation:** The program will not make use of discovered vulnerability information or perform destructive attacks. Discovered information will only be reported to users.
- **Real-World Targeting:** The system will not scan infrastructure outside of the controlled environment.
- **Internal Network Monitoring:** The system will not monitor or analyze a machine's internal activities.

- **Identity Management:** Identity management precautions, such as user authentication and access control, are not included within the system.
- **Compliance or Risk Scoring Engine:** The system will not produce compliance or business risk reports based on new or existing vulnerabilities identified.
- **Enterprise-Level Performance:** This project is not intended to meet enterprise-scale availability or internet-scale performance standards.

1.3 Definitions and Acronyms

The following definitions and acronyms are provided to explain terminology used throughout the document:

- **Attack Surface Management (ASM):** the process of continuously discovering and monitoring all exposed digital assets in an organization's attack surface.
- **Attack Surface:** all exposed digital assets, such as IP addresses and servers, domains and subdomains, APIs, cloud resources, and open ports.
- **Asset:** property owned by an individual or organization that has value.
- Common Vulnerabilities and Exposures (CVE): a list of publicly disclosed cybersecurity vulnerabilities.
- **National Vulnerability Database (NVD):** U.S government-maintained database that provides information on CVEs
- **Active Probing:** the process of sending network requests to services to test their behavior and acknowledgements.
- **Port Scanning:** the process of discovering open ports to verify accessible services through the given port.
- **Red Phase:** A controlled system that simulates an attack environment by introducing vulnerabilities to test the function of the ASM.
- **Container:** an isolated runtime environment that wraps an application and any dependencies.
- **Virtual Machine (VM):** a virtual computer environment that runs any specified operating system that acts as a physical machine.
- **Common Vulnerabilities and Exposures (CVE):** a list of public security flaws in computing.
- **Domain Name System (DNS):** how a computer looks up and find corresponding IP addresses with their domain names.
- **CSV:** a plaintext file that stores tabular data in a format separated by commas
-

1.4 Assumptions

The following assumptions define the conditions needed to be met for the project's requirements to be successfully implemented:

- The ASM engine will have network access to all systems, services, and ports in the target environment.
- The target environment will consist of multiple containers within a single virtual machine on a single internal network.
- Network scanning will be sufficient to identify exposed services, assets, and system configuration changes.
- The DNS will work properly between all systems.

- Any changes made to the controlled environment will be visible through scheduled scans.
- Asset information and results from scans will be scaled appropriately and stored in simple files or basic databases.
- Logins will not be required as authorized members will already have access to the system.

The project requirements assume that these conditions will remain true throughout system implementation.

1.5 References

The following are the resources that guided our requirements specifications:

Am, M. I. (2025, October 21). *Amass 5.0.0 Usage for Recon*. Medium. https://medium.com/@marduk.i.am/amass-5-0-0-usage-for-recon-8041bc727480

amass | Kali Linux Tools. (n.d.). Kali Linux. https://www.kali.org/tools/amass/

Anamar Taknev. (2024, August 5). *Beginners guide to amass - Hackercool Magazine*. Hackercool Magazine.https://www.hackercoolmagazine.com/beginners-guide-to-amass/?srsltid=AfmBOooWfPNIBIO9HpFRqYTWkQaEiNpVSZpMLk6j5pIWy6FBBZcp_182

Nmap. (2019). *Chapter 15. Nmap reference guide | nmap network scanning*. Nmap.org. https://nmap.org/book/man.html

Nmap. (2019). *Chapter 15. Nmap reference guide | nmap network scanning*. Nmap.org. https://nmap.org/book/man.html

nmap | Kali Linux Tools. (n.d.). Kali Linux. https://www.kali.org/tools/nmap/

Ojha, Y. (2023). *reNgine*. Rengine.wiki. https://rengine.wiki/

OWASP Amass Contributors. (2017). *OWASP Amass Project*. Github.io. https://owasp-amass.github.io/docs/

Running Nuclei - ProjectDiscovery Documentation. (2020). Projectdiscovery.io; ProjectDiscovery Documentation. https://docs.projectdiscovery.io/opensource/nuclei/running

Watchtower. (2025). Containrrr.dev. https://containrrr.dev/watchtower/

Ojha, Y. (2023). *reNgine*. Rengine.wiki. https://rengine.wiki/

2.0 Constraints

Because this project is a prototype, there are limitations and conditions that constrain how the system can be designed and operated. These constraints are noted to ensure the project remains within the scope described while still meeting the functional requirements.

2.1 Environment

The environment of this system refers to the network, infrastructure, and the setting in which the system will be working. For this project, we know the system must operate in a controlled environment to simplify implementation and limit external interference. Additionally, the environment has to support basic DNS resolution between systems. It is also important that the ASM engine can communicate with the virtual machine and all containers over an internal network to perform scans, discover open ports, and detect vulnerabilities. To ensure reliable scanning, we want to ensure that all target systems are on the same network segment. Lastly, the environment must only allow authorized access to the target systems.

2.2 System

System constraints refer to the technical and design aspects of the project. These constraints include the need to rely on network-based scanning techniques rather than host-based scanning because the target system will be running on a virtual machine. The system must integrate with open-source tools for vulnerability detection. Another constraint is that the system must be transferable within a shared virtual machine or a Dockerized service. This creates an environment that is consistent across users and allows the system to be recreated or moved if something breaks. Lastly, the system needs to provide a dashboard for vulnerabilities and scans and operate without requiring user authentication.

3.0 Functional Requirements

3.1 Asset Discovery

3.1.1 The system shall scan the target network for IP addresses and newly opened ports.

3.1.2 The system shall enumerate the subdomains of the target's domain.

3.1.3 The system shall keep an updated inventory of all newly discovered assets behind ports

3.1.4 The system shall scan for new services or processes on the target network.

3.2 Active Probing

3.2.1 The system shall send protocol specific requests to services.

3.2.2 The system shall validate responses against known responses

3.2.3 The system shall flag for any suspicious responses

3.3 Vulnerability Detection

3.3.1 The system shall detect configuration drift and other new vulnerabilities in the target environment.

3.3.2 The system shall compare found vulnerabilities with CVE identifiers when available.

3.3.3 The system shall store CVE comparisons and vulnerability detections for historical comparison.

3.4 Reporting & Notifications

3.4.1 The system shall generate attack surface reports and drift reports highlighting any changes since the previous scans.

3.4.2 The system shall store all reports in a CSV format file of all drift reports and known CVEs.

3.5 Dashboard & GUI

3.5.1 The system dashboard shall provide a dashboard to view a mapping of assets.

3.1.2 The system dashboard shall show a view of previous scans and what tools were used in these scans.

3.5.3 The system dashboard shall show a view of active vulnerabilities with various severities.

4.0 Non-Functional Requirements

4.2 Security

4.2.1 The system shall operate only in authorized target environments

4.2.2 The system shall make sure the target environment is not accessible over the public internet without safeguards.

5.0 Target Environment Requirements

The target environment is what the ASM system will be tested on, and it has necessary requirements for the evaluation of the ASM system.

5.1 Target Infrastructure

5.1.1 The target environment must include multiple containers representing various services (e.g., web servers, database servers)

5.2 Network Configuration

5.2.1 The target environment containers shall be on the same internal network.

4.2.2 The target environment shall support DNS server with DNS resolution between containers.

5.3 Vulnerability Injection

5.3.1 The target environment must include vulnerable configurations that the system can detect.

6.0 External Interface Requirements (use if applicable)

6.1 User Interface Requirements

We will need to make a dashboard that allows us to take the vulnerabilities found through our scans so that we can easily see what we are working with. We will need it to be able to interact with the

database and show things like CVSS scores, any matches to CVEs that we found on the internet or through using APIs, and drift reports that were created in each scan. We could even possibly have it give a visual representation of the network that can help visualize where vulnerabilities are coming from. Should be able to start scans from the dashboard and have them be able to run automatically.

6.2 Hardware Interface Requirements

All the VMs, Docker containers, and devices that are loaded on need to be able to access each other through the network, with any of our containers in our enclave, we are defending being on the same network. We will need some sort of database to store the information we have collected about CVEs, vulnerabilities, and our drift reports.

6.3 Software Interface Requirements

It will be necessary for our containers and VMs to be on Linux so that we have better access and flexibility to customize what our operating systems and network look like. We will need each to be part of DNS network. We will need reNgine to package the other tools we will be using, OWASP and Nmap together for easier access and use. We will need to run scripts to create our drift reports as well as access APIs that allow us to pull and compare CVEs with our found vulnerabilities.

6.4 Communication Interface Requirements

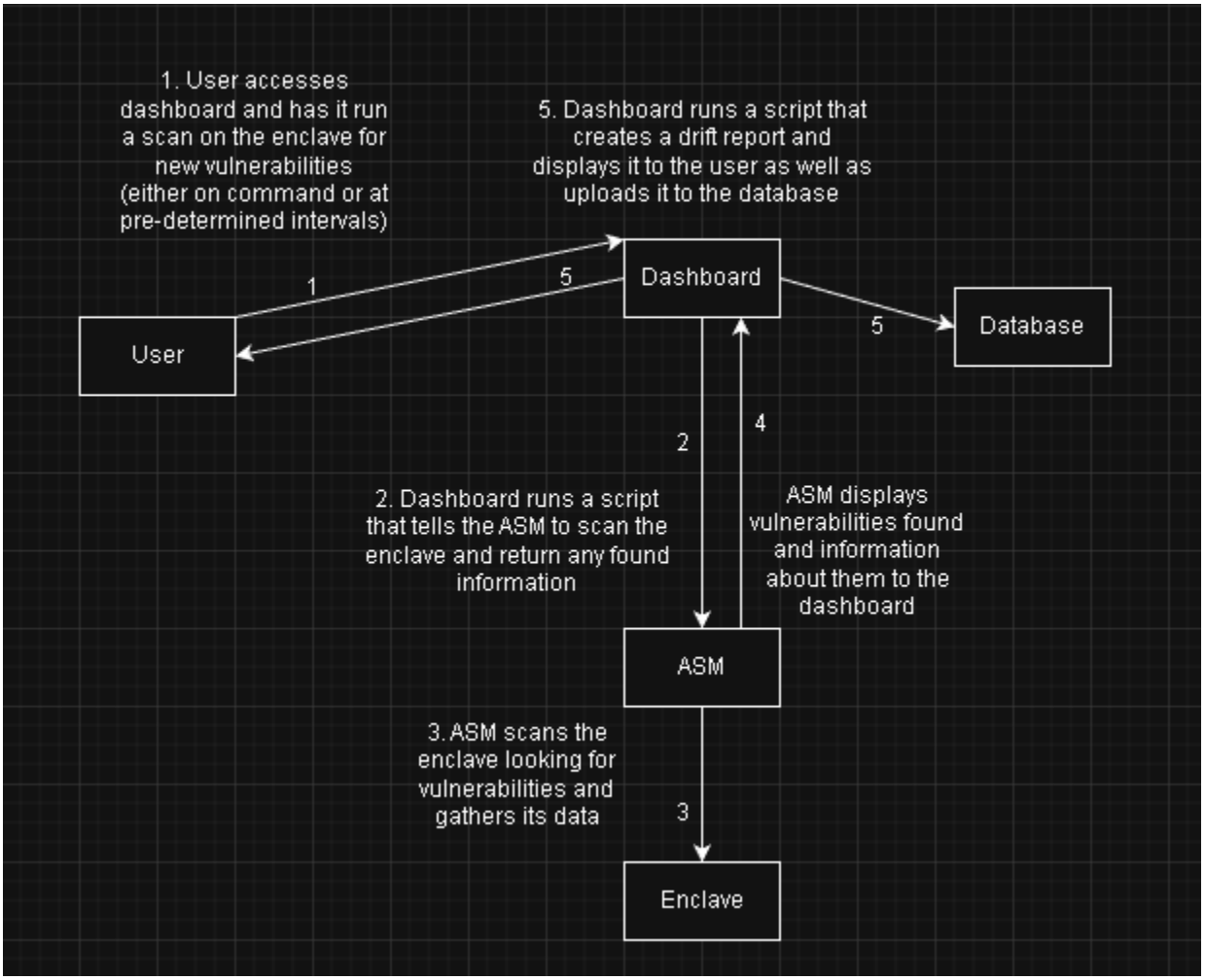
We will need every VM and container we make to be able to access each other through the network so that we can set up our scanners and test that they work.

7.0 Analysis

7.1 Use Cases

The main use case for our ASM will be to scan a local network for any changes, such as newly opened and unused ports, or suspicious traffic, and alert them. The system will be able to store these vulnerabilities, give them a CVSS score potentially, compare them to CVEs, and compile reports on what has been found to allow the user to know what needs to be looked out for and fixed. The purpose of the ASM is to alert on possible vulnerabilities so that the user can fix them.

7.2 Data Flow


1. User accesses dashboard and has it run a scan on the enclave for new vulnerabilities (either on command or at pre-determined intervals)
5. Dashboard runs a script that creates a drift report and displays it to the user as well as uploads it to the database
1
5
Dashboard
User
5
Database
2
4
2. Dashboard runs a script that tells the ASM to scan the enclave and return any found information
ASM displays vulnerabilities found and information about them to the dashboard
ASM
3. ASM scans the enclave looking for vulnerabilities and gathers its data
3
Enclave